



FACULTAD DE
INGENIERÍA



UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY

Tarea 2

Introducción a la Ciencia de Datos

Edición 2026

Integrantes:

Tomás Alejo Spoturno Gonzalez, 5.165.789-6

Nicolás Buero, 5.000.105-4

Universidad de la República
Facultad de Ingeniería

Junio 2026

Índice

1. Introducción	2
2. Parte 1: Dataset y representación numérica de texto	2
2.1. Preparación del dataset	2
2.2. Verificación del balance de clases	3
2.3. Representación Bag of Words	3
2.4. Representación TF-IDF	4
2.5. Visualización PCA sobre TF-IDF	5

1. Introducción

Este informe documenta la resolución de la Tarea 2 del curso *Introducción a la Ciencia de Datos* (edición 2026). La tarea es continuación directa de la Tarea 1 y utiliza el mismo dataset: un subconjunto muestreado del corpus abierto *All the News 2.1 - Component One*, que contiene artículos de noticias publicados en distintos medios de prensa. El objetivo de esta entrega es **construir y evaluar clasificadores automáticos de texto** que identifiquen a qué medio pertenece cada artículo, partiendo de representaciones numéricas del texto.

El trabajo se estructura en dos partes. La Parte 1 cubre la preparación de los datos, la representación numérica del texto mediante *Bag of Words* y TF-IDF, y una exploración visual con PCA. La Parte 2 cubre el entrenamiento, la validación y la comparación de modelos de clasificación.

El código que genera los resultados está en el notebook `Tarea_2/2026_tarea2.ipynb` del repositorio público <https://github.com/Nicolasvb23/intro-cd>.

2. Parte 1: Dataset y representación numérica de texto

2.1. Preparación del dataset

El proceso de limpieza de datos y la función `clean_text` son los mismos que en la Tarea 1, a los que se remite para el detalle. En resumen, sobre el dataset original de 30.213 artículos se eliminaron 141 filas con `publication` nula, 1.176 con `article` nulo o placeholder, y 13 duplicados exactos, resultando en **29.024 artículos limpios**. La función `clean_text` aplica los mismos pasos que en la entrega anterior (eliminación de cabecera, minúsculas, URLs, puntuación, números aislados y normalización de espacios), sin agregar filtrado de *stopwords* ni identificadores de medios en esta etapa, ya que esas decisiones se toman a nivel del vectorizador y son parte del análisis de la Sección 2.5.

Selección de los tres medios principales

El análisis de esta tarea se restringe a los **tres medios con mayor cantidad de artículos** en `df_clean`. Los recuentos se muestran en la Tabla 1.

Cuadro 1: Top 3 medios de prensa por cantidad de artículos en el dataset limpio.

Medio	Artículos	% del total (top 3)
Reuters	9.266	63,2 %
The New York Times	2.805	19,1 %
CNBC	2.578	17,6 %
Total top 3	14.649	100 %

Los tres medios coinciden con los tres primeros del top 5 de la Tarea 1. El dataset resultante presenta un **desbalance pronunciado**: *Reuters* acumula el 63,2 % de los artículos, frente al

19,1 % de *NYT* y el 17,6 % de *CNBC*. Esto tiene implicancias sobre las métricas de evaluación que se discuten en la Parte 2.

Partición train/test

El dataset de los tres medios se dividió en un conjunto de **entrenamiento (70 %)** y un conjunto de **test (30 %)**, utilizando muestreo estratificado por medio de prensa (`stratify=y`). La estratificación garantiza que la proporción de artículos de cada medio sea la misma en ambos conjuntos. Se fijó `random_state=23` para reproducibilidad.

2.2. Verificación del balance de clases

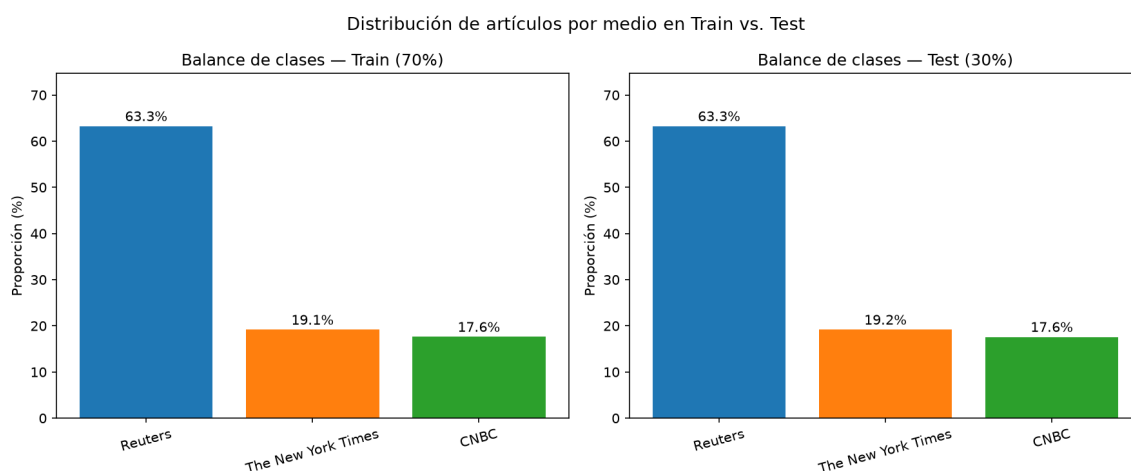


Figura 1: Proporción de artículos de cada medio en los conjuntos de entrenamiento (izquierda) y test (derecha). La estratificación garantiza distribuciones idénticas en ambos conjuntos.

La Figura 1 confirma que las proporciones son idénticas en train y test, como se espera de un split estratificado. El desbalance del dataset se preserva de forma exacta: aproximadamente 62 % de los artículos corresponden a *Reuters*, lo que implica que un clasificador trivial que siempre prediga “Reuters” alcanzaría un *accuracy* de ese orden sin aprender nada del contenido. Este punto se retoma en la Parte 2 al discutir las limitaciones del *accuracy* como única métrica.

2.3. Representación Bag of Words

Cómo funciona

La representación *Bag of Words* (BoW) convierte cada documento en un vector numérico de la siguiente manera: se construye un **vocabulario** fijo con todas las palabras distintas que aparecen en el corpus de entrenamiento, y cada documento se representa como un vector de tamaño igual al vocabulario donde la posición j contiene la **cantidad de veces** que la palabra j aparece en ese documento. El orden de las palabras dentro del texto se ignora completamente.

Tamaño de la matriz

Se ajustó un `CountVectorizer` con parámetros por defecto sobre el conjunto de entrenamiento. La matriz resultante tiene dimensiones $[n_docs \times n_palabras]$, donde:

- **Filas:** 10.254 artículos en el conjunto de entrenamiento.
- **Columnas:** 84.358 palabras distintas en el vocabulario aprendido sobre el train.

Por qué es una matriz *sparse*

Si se almacenara como una matriz densa (todos los valores en memoria), la matriz BoW ocuparía aproximadamente:

$$10,254 \times 84,358 \times 8 \text{ bytes} \approx 6,920 \text{ MB}$$

Esto sería inmanejable incluso con este dataset reducido. Sin embargo, la gran mayoría de las entradas son cero: la densidad de la matriz es del 0,247 %, lo que significa que más del 99 % de sus valores son cero. Ninguna nota usa más de unos cientos de palabras distintas, mientras que el vocabulario total tiene decenas de miles. Las matrices *sparse* almacenan únicamente los valores no-nulos y sus posiciones (formato CSR), reduciendo el uso de memoria de 6.920 MB a tan solo 25,7 MB.

Si en lugar de 14.000 artículos tuviéramos el dataset completo de 1,8 millones de artículos del corpus original y un vocabulario de cientos de miles de palabras, la matriz densa sería completamente inviable en cualquier equipo de cómputo estándar.

2.4. Representación TF-IDF

Qué es un n-grama

Un **n-grama** es una secuencia contigua de n tokens (palabras o caracteres). Un unigrama ($n = 1$) es una palabra individual; un bigrama ($n = 2$) es un par de palabras consecutivas. Por ejemplo, a partir del texto “the new york times” se extraen los bigramas “the new”, “new york” y “york times”. Incluir bigramas en el vocabulario permite capturar expresiones compuestas y contexto local que los unigramas no pueden representar. El parámetro `ngram_range=(1,2)` en el vectorizador combina unigramas y bigramas en el mismo vocabulario, aunque a costa de aumentar dramáticamente el número de columnas.

La transformación TF-IDF

El principal problema de BoW es que las palabras muy frecuentes en todo el corpus (artículos, preposiciones, conjunciones) dominan los vectores aunque no aporten información discriminante entre medios. TF-IDF (*Term Frequency - Inverse Document Frequency*) corrige esto ponderando cada conteo por la siguiente expresión:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

donde $\text{TF}(t, d)$ es la frecuencia normalizada del término t en el documento d , e $\text{IDF}(t) = \log\left(\frac{N+1}{n_t+1}\right) + 1$ con N el número total de documentos y n_t la cantidad de documentos que contienen t . La transformación tiene dos efectos:

- **Penaliza términos ubicuos:** una palabra que aparece en todos o casi todos los documentos tiene $\text{IDF} \approx 0$, por lo que su peso TF-IDF cae a cero o próximo a cero, independientemente de cuántas veces aparezca en un documento dado.
- **Premia términos discriminantes:** una palabra rara que aparece mucho en un documento particular y poco en el resto recibe un peso alto.

El vectorizador `TfidfVectorizer` se ajusta únicamente sobre el conjunto de entrenamiento (`fit_transform`). El vocabulario y los valores IDF aprendidos se aplican luego sobre el conjunto de test (`transform`), sin recalculr nada sobre el test. Esto simula correctamente el escenario real donde el modelo no tiene acceso a los datos de evaluación al momento de entrenarse.

2.5. Visualización PCA sobre TF-IDF

Consideraciones de implementación

El análisis de componentes principales (*PCA*) requiere materializar la matriz TF-IDF como densa para poder centrarla. Con el vocabulario completo de 84.358 palabras y 10.254 documentos de entrenamiento, esto supone aproximadamente 6,9 GB en RAM. En esta tarea se optó por utilizar directamente `sklearn.decomposition.PCA`, materializando la matriz completa con `.toarray()`. Como alternativa sin restricciones de memoria, `TruncatedSVD` opera sobre la representación *sparse* sin densificarla, aunque sin centrar los datos previamente.

Resultados con la configuración base

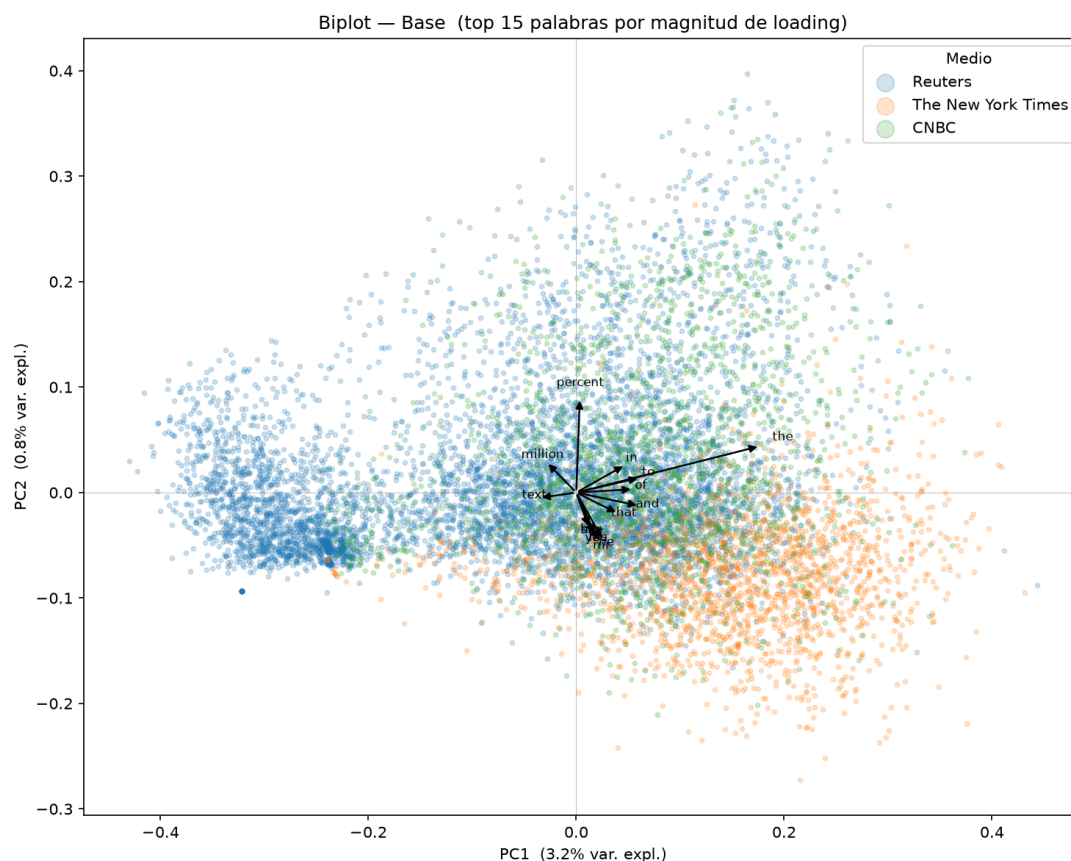


Figura 2: Biplot sobre TF-IDF base: puntos coloreados por medio de prensa y flechas con las 15 palabras de mayor loading en el plano PC1-PC2.

La Figura 2 muestra el conjunto de entrenamiento proyectado sobre las dos primeras componentes principales. Se observa que *Reuters* forma una nube claramente separada del resto, especialmente a lo largo de la primera componente. *NYT* y *CNBC* presentan mayor superposición entre sí, aunque con cierta separación. Las dos primeras componentes explican solo el 4,1 % de la varianza total, lo que indica que **no es posible separar los tres medios de forma perfecta con únicamente 2 componentes**: los vectores TF-IDF viven en un espacio de alta dimensionalidad y la proyección bidimensional pierde la mayoría de la información. Las flechas superponen los vectores de *loading* de las 15 palabras con mayor magnitud; su longitud indica cuánto contribuye esa palabra y su dirección hacia qué lado del plano “apunta” esa palabra.

La Figura 2 muestra que las flechas de mayor loading apuntan mayoritariamente hacia el lado **positivo** de PC1, que es donde se concentran *NYT* y *CNBC*. *Reuters* queda en el extremo **negativo** de PC1, separado del resto. Esto indica que los términos con mayor peso en la primera componente son palabras más presentes en *NYT* y *CNBC*; los artículos de *Reuters* se caracterizan por tener valores bajos para esas dimensiones. Los labels se superponen y son difíciles de leer individualmente, lo cual es consistente con que varios términos contribuyen de forma similar a PC1 sin que uno solo domine claramente.

Comparación de configuraciones

Se analiza el efecto de tres modificaciones al `TfidfVectorizer`. Para cada configuración se muestra el biplot correspondiente.

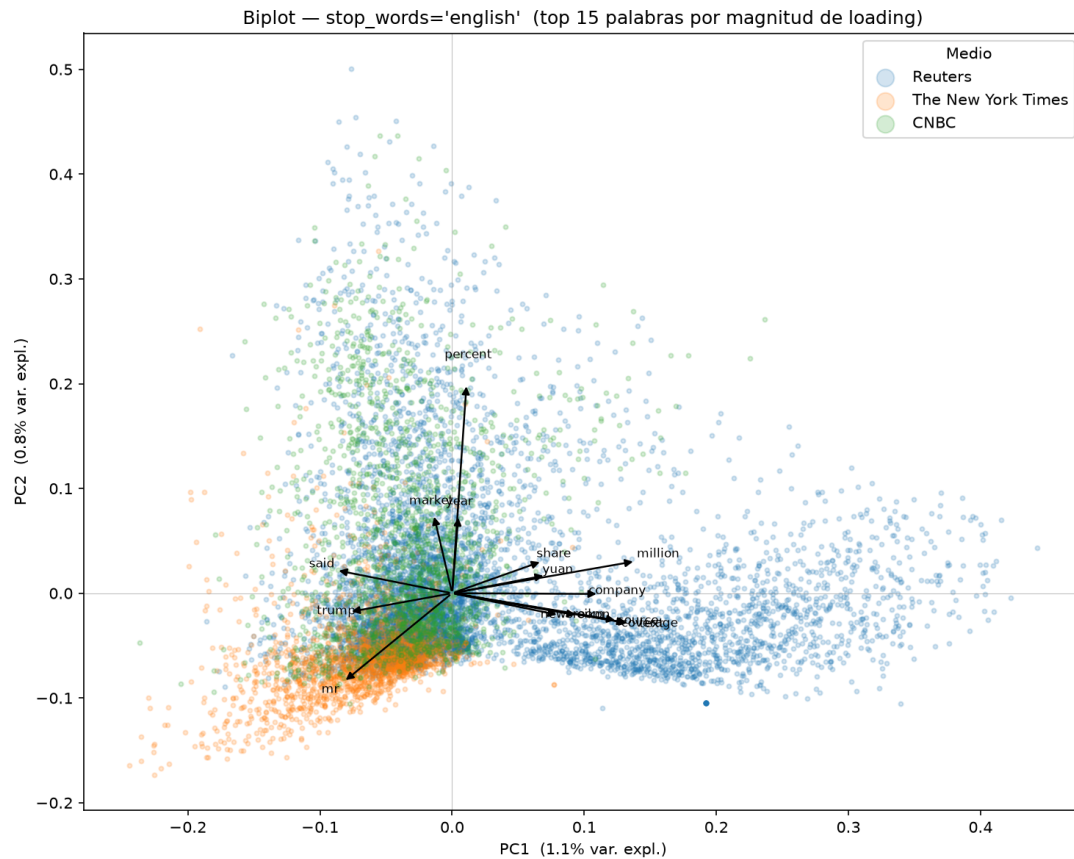


Figura 3: Biplot con `stop_words='english'`: al eliminar palabras funcionales, los loadings quedan dominados por términos de contenido. PC1 explica 1,1 % de la varianza.

a) Filtrado de *stop words* (`stop_words='english'`). Al eliminar las palabras funcionales del inglés (artículos, preposiciones, conjunciones), palabras como “*the*”, “*of*” o “*in*” desaparecen del vocabulario y no pueden aparecer como flechas en el biplot. En la Figura 3 los términos de mayor loading visibles son “*reuters*” y “*percent*”, que son palabras de contenido características de ese medio. Contra lo intuitivo, PC1 cae de 3,2 % a 1,1 %: al eliminar los términos más frecuentes se reduce la concentración de varianza en la primera dirección, distribuyéndola entre más componentes.

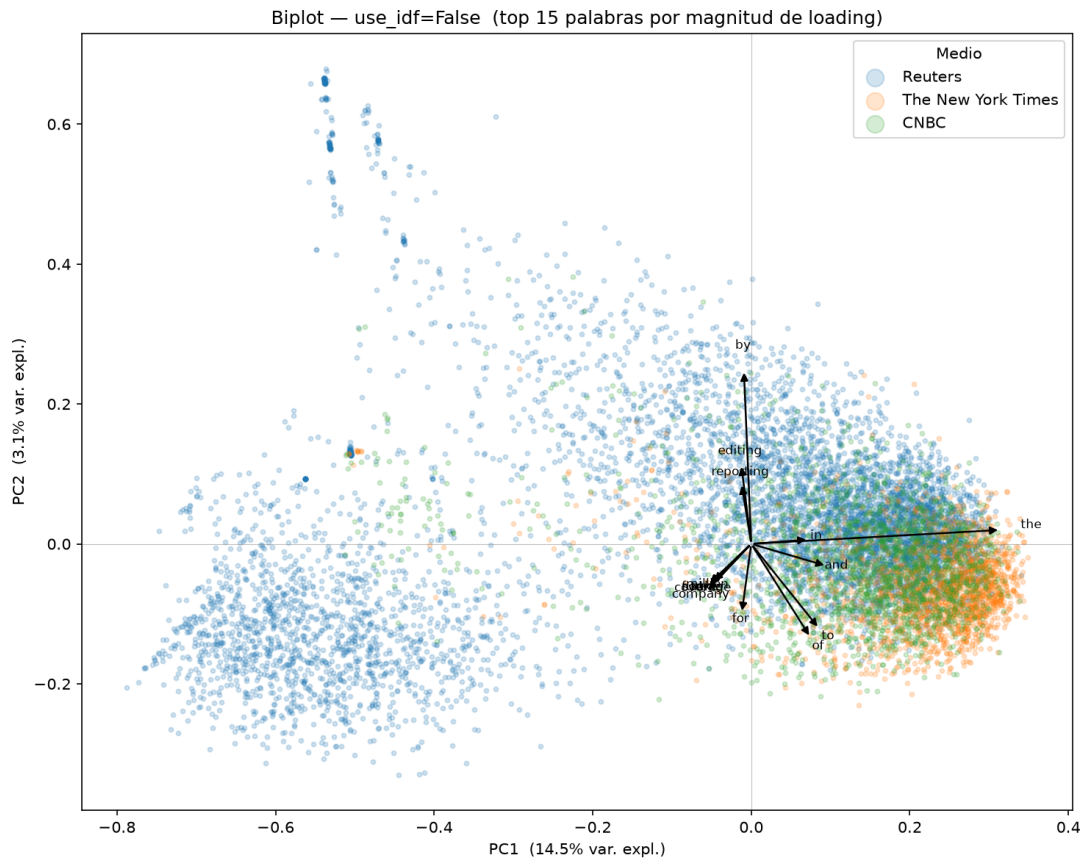


Figura 4: Biplot con `use_idf=False`: una flecha larga domina PC1, correspondiente a *“the”*. PC1 explica 14,5 % de la varianza.

b) Sin IDF (`use_idf=False`). Con IDF desactivado, las palabras muy comunes recuperan el peso que el factor inverso les quitaba. El biplot (Figura 4) ilustra el problema con claridad: aparece una única flecha dominante, notablemente más larga que el resto, correspondiente a *“the”*. PC1 salta a 14,5 % de varianza explicada (frente al 3,2 % de la configuración base) porque ahora la primera componente captura principalmente variación en la frecuencia de ese artículo, que es equidistribuida entre los tres medios y no aporta discriminación. Esto ilustra concretamente por qué el factor IDF es fundamental en clasificación de texto.

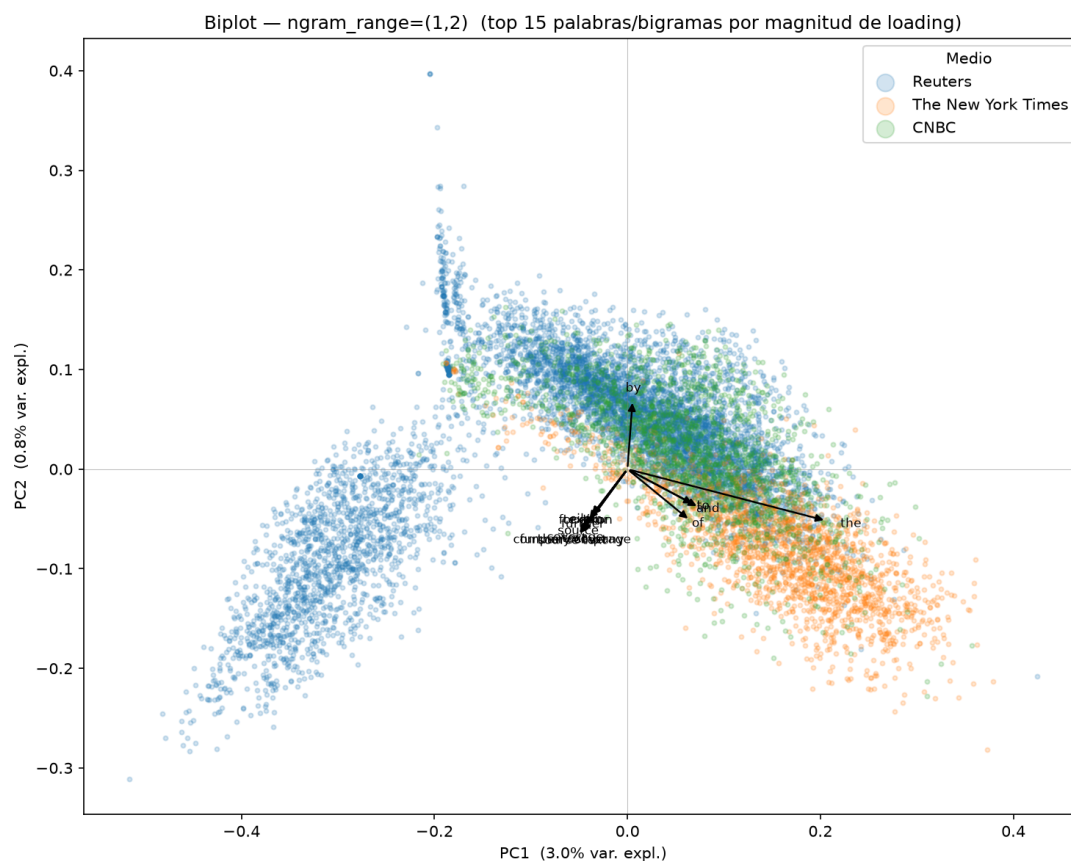


Figura 5: Biplot con `ngram_range=(1,2)` y `max_features=60.000`: el vocabulario incluye secuencias de dos palabras consecutivas. PC1 explica 3,0 % de la varianza.

c) **Bigramas** (`ngram_range=(1,2)`). Al agregar bigramas el vocabulario incluye todas las secuencias de dos palabras consecutivas, lo que lo hace crecer hasta cientos de miles de términos. Para hacer viable la densificación se acotó a `max_features=60.000`. El biplot (Figura 5) conserva una estructura similar a la configuración base: *Reuters* aparece en el extremo negativo de PC1 y “*reuters*” figura entre los términos de mayor loading. La varianza explicada por PC1 (3,0%) es prácticamente igual a la base (3,2%), lo que indica que la adición de bigramas no altera cualitativamente la separación en las primeras dos dimensiones.

Varianza explicada

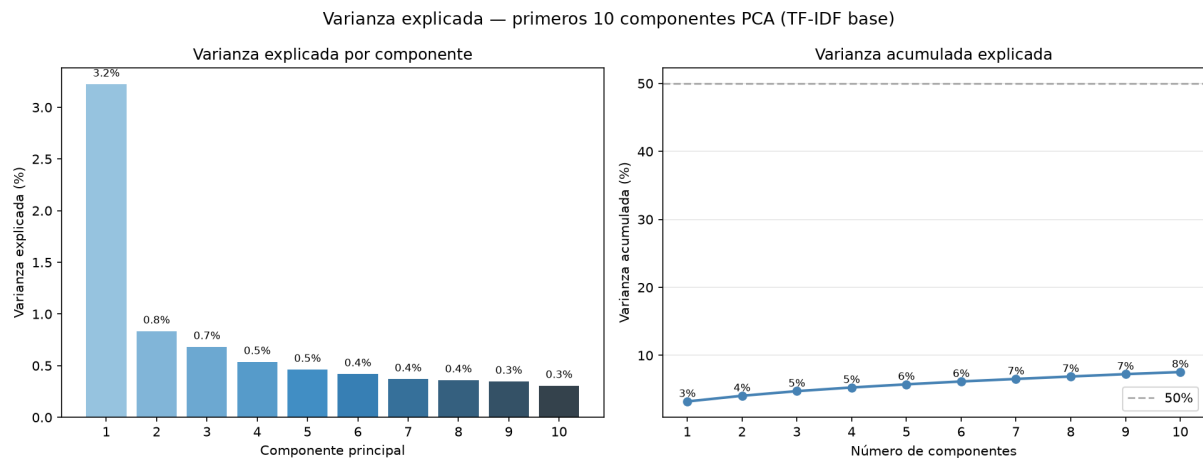


Figura 6: Varianza explicada por cada componente principal (izquierda) y varianza acumulada (derecha) para los primeros 10 componentes sobre la configuración TF-IDF base.

La Figura 6 muestra que la varianza explicada decrece rápidamente con el número de componentes y que incluso con 10 componentes la varianza acumulada es relativamente baja. Esto refleja la **alta dimensionalidad intrínseca del texto**: la información está repartida entre miles de términos, y ninguna dirección captura una fracción grande de la varianza total.

Origen de la separación observada